

Rules of Engagement (RoE) — NLT AI Chatbot Pentest

Purpose: authorize and define a safe, auditable engagement to assess the security of the NLT (NodeLine Tech) customer-support chatbot and its AI-specific components (Flask web interface, RAG system, vector database, autonomous AI agent, and admin interfaces) during pre-launch.

Note: This RoE is specifically tailored for the NLT Chatbot system that is in its testing phase. For the purpose of testing the tester will be given development source code and launch scripts/documentation so they can run their own development environment for testing and evaluation.

1. Parties & Authorization

Tester (Consultant): [Penetration Testing Company/Individual]

Client (Organization): NodeLine Tech (NLT) - AI Chatbot Division

Authorization: NLT authorizes the named Tester to perform AI-specific security testing against the NLT Chatbot system as defined below. This RoE is a binding agreement **only** for the named Tester(s) and for the specified timeframe. Tester must carry written authorization when performing tests.

Signature (Tester): _____ **Date:** _____

Signature (NLT Representative): _____ **Date:** _____

2. Engagement Overview & Objectives

Primary goal: Identify AI-specific security weaknesses in the NLT chatbot system that could lead to data leakage, prompt-injection/jailbreak, harmful or false outputs, RAG poisoning, or abuse of the autonomous AI agent and admin interfaces.

Objectives:

- Map and Fingerprint AI capabilities and underlying model(s).
 - Test for system-prompt leakage, prompt injection/jailbreaking bypassing AI security levels (1-5), harmful/off-topic output, RAG poisoning, AI agent manipulation, and PII/confidential information leakage.
 - Evaluate security of role-based access controls, specifically as they pertain to the AI chatbot and its access.
 - Test autonomous AI agent decision-making for ticket manipulation and privilege escalation.
 - Assess vector database security and knowledge base information disclosure.
 - Provide prioritized findings, reproducible PoCs (sanitized), mitigation recommendations, and a retest verification plan.
-

3. Time Window & Scheduling

Test window (local time): [Start date/time] → [End date/time]

Planned test days/hours: Business hours recommended (09:00–17:00 local time) for technical support availability

System availability: NLT Chatbot system can be run locally by the tester via Docker containers - testing can occur outside business hours

Maintenance windows to avoid: N/A

Monitoring plan: System health available via `/api/health` endpoint - automated monitoring of Flask and Ollama service status during testing

4. Defined Scope

In-scope (NLT chatbot-focused)

- **Core AI Chat Interface:** `POST /api/chat` (main chat endpoint with RAG and multi-level security)
- **Conversation Management:** `GET /api/conversation/<id>`, `POST /api/conversation/<id>/clear`, `GET /api/conversations`
- **Authentication System:** `POST /api/login`, `POST /api/register`, `GET /api/user`, `POST /api/logout` (session-based auth with CSRF)
- **Support Ticket System:** `POST /api/tickets/create`, `GET /api/tickets/<ticket_number>`, `POST /api/tickets/<id>/update`, `GET /api/tickets/user`
- **RAG Knowledge Base:** `GET /api/knowledge-base/stats`, `POST /api/knowledge-base/search` (admin-only), `POST /api/knowledge-base/reindex` (admin-only)
- **Vector Database (ChromaDB):** Testing semantic search, document retrieval, and embedding injection via knowledge base
- **Autonomous AI Agent:** Ticket decision-making system that auto-closes, escalates, or offers discounts based on conversations
- **Admin Interfaces:** `GET /api/admin/tickets`, `PUT /api/admin/tickets/<id>/assign`, `PUT /api/admin/tickets/<id>/status`, `POST /api/admin/tickets/<id>/reply`
- **System Health:** `GET /api/health`, `GET /api/health/ollama`, `GET /api/models`, `GET /api/configured-model`
- **Web Interface Routes:** `/`, `/chat`, `/tickets`, `/admin`, `/products` and associated static files
- **AI Security Levels:** Testing bypass of levels 1-5 security filtering (pattern-based, AI-powered input/output analysis)

Out-of-scope (unless separately authorized in writing)

- Physical infrastructure, Docker host system, or hardware-level attacks
- Docker Compose files, launch.sh, stop.sh and any other utility scripts
- Ollama AI Model Local or Cloud LLM inference endpoints (default port 11434, configurable via OLLAMA_BASE_URL)
- Ollama model files or container filesystem modifications (testing limited to chatbot API interactions)
- Model Extraction (not required as "off-the-shelf" open source models with out fine-tuning are used)
- SQLite database file manipulation (testing via API endpoints only)
- ChromaDB vector database files (testing via application interface only)
- Docker container escape or privilege escalation outside application scope
- Destructive actions against the SQLite database or vector database files
- Social-engineering attacks targeting staff (unless separately authorized)
- Tests that may materially degrade service availability (see DoS restrictions)
- Network-level attacks against Docker containers or host networking

Note: The purpose of providing the launch infrastructure-as-code is to allow the tester to control their environment and is not intended to be used as part of the penetration test as it will not be present in production.

5. Allowed Tests & Constraints

The following tests are allowed **within the specified scope and constraints**:

Permitted (low → medium risk)

- **Passive reconnaissance:** cataloging Flask endpoints, analyzing client-side JavaScript, parameter enumeration, session analysis
- **Active functional probing:** sending chat messages, replaying/mutating API requests, testing conversation management, measuring response metadata
- **AI Security Level bypassing:** testing prompt injection and jailbreak techniques against security levels 1-5 using role-play, encoding, multi-turn chains
- **Authentication testing:** session manipulation, CSRF bypass attempts, role escalation testing (customer → admin)
- **RAG system testing:** knowledge base information disclosure, vector search manipulation
- **Autonomous AI Agent manipulation:** attempting to influence ticket decisions (close, escalate, discount) through conversation manipulation
- **Admin interface testing:** unauthorized access attempts, privilege validation, ticket manipulation via admin endpoints
- **Model extraction (limited):** controlled question-answer collection up to query limits for analysis (no bulk data exfiltration)
- **Knowledge base enumeration:** testing access to development documents, internal keys, configuration data via RAG system

High-risk tests (allowed only in development environment)

- **ChromaDB index poisoning:** injecting malicious documents or embeddings into the vector database through knowledge base modification
 - **Production data destruction:** deleting conversations, tickets, user accounts, or vector database contents
 - **Bulk data extraction:** high-volume API calls designed to extract entire databases, conversation histories, or knowledge base
 - **Distributed DoS attacks:** coordinated requests designed to overwhelm Flask application, Ollama service, or ChromaDB
 - **File system access:** attempting to read/write application files, configuration files, or Docker volumes outside API scope
-

6. Admin Interfaces & Sensitive Endpoints

The following admin endpoints are **in-scope** for authorized security testing:

- `/api/admin/tickets` - Admin ticket management (test authorization bypass)
- `/api/admin/tickets/<id>/assign` - Ticket assignment (test privilege escalation)
- `/api/admin/tickets/<id>/status` - Status updates (test unauthorized modification)
- `/api/admin/tickets/<id>/reply` - Admin responses (test impersonation)
- `/api/knowledge-base/reindex` - Vector DB rebuild (admin-only, test access controls)
- `/api/knowledge-base/search` - RAG testing interface (admin-only, test data disclosure)

Sensitive knowledge base documents testing disclosure of these via RAG system is **permitted** as this represents a realistic attack scenario.

7. Test Data, Synthetic Canaries & Test Environment

- **Vector Database:** Testing limited to existing ChromaDB collection - no modification of vector database files or creation of test indices required
 - **Synthetic canaries:** Use unique strings like `CANARY_NLT_[TESTER_INITIALS]_[YYYYMMDD]` for RAG membership inference tests.
Example: `CANARY_NLT_AB_20261022`
 - **Test accounts:** Pre-existing test accounts available:
 - Customer: `customer@example.com` / `customer123` (standard user role)
 - Admin: `admin@nodelinetech.com` / `admin123` (admin role with full privileges)
 - **Test tickets:** Create synthetic support tickets for AI agent manipulation testing - use test customer account for ticket creation
 - **Knowledge base testing:** Attempt to retrieve sensitive documents from `/knowledge_base/development/` containing API keys and credentials via RAG system
-

8. Rate Limits, Query Caps & Load Constraints

To avoid accidental service disruption, the Tester agrees to the following limits unless using their development environment:

- **Chat API (/api/chat):** max 30 requests per minute (application enforced)
- **Login attempts (/api/login):** max 5 requests per minute (application enforced)
- **General API endpoints:** max 100 requests per hour (application enforced)
- **Admin endpoints:** max 200 requests per hour (application enforced)
- **Ollama inference:** respect model response times, avoid overwhelming local inference
- **Total model extraction queries:** max 200 queries per day for analysis purposes
- **Vector search operations:** max 50 queries per hour to avoid ChromaDB performance impact Any need to exceed these caps must be requested in writing and approved by the Client before execution.

9. Data Handling, Sensitive Findings & Remediation Coordination

- **Discovery of credentials/secrets:** If Tester successfully extracts sensitive credentials (API keys, passwords, certificates) from the RAG system or other sources, Tester MUST:
 1. Document the finding with exact reproduction steps
 2. Notify NLT security contact immediately if credentials appear to be production-valid
 3. NOT use discovered credentials for unauthorized access to external services
 4. Include findings in the security report with remediation recommendations
 5. Note: Discovery of test/development credentials in /knowledge_base/development/ is expected and part of the assessment scope
- **Evidence storage:** Test artifacts (API requests/responses, chat logs, screenshots, extracted data) will be stored encrypted and shared securely with NLT. No public disclosure without written approval.
- **Data retention:** Tester will retain evidence for 90 days after final report delivery for retest validation purposes, then securely delete all NLT-related test data.

10. Logging, Monitoring & Alerting

- **Application logging:** Flask application logs all authentication attempts, API calls, and security violations - logs available for test correlation
 - **Health monitoring:** System health endpoints (`/api/health` , `/api/health/ollama`) provide real-time status of components during testing
 - **Rate limiting alerts:** Application enforces rate limits that may temporarily block testing - this is expected behavior and part of the security assessment
 - **AI security alerts:** Security levels 2+ log blocked attempts - these logs will help validate security control effectiveness
 - **Docker monitoring:** Container health checks monitor Flask and Ollama service availability throughout testing period
 - **No external alerting:** System is typically deployed locally/privately - no external security monitoring services to coordinate with
-

11. Emergency Escalation, Pause & Stop Conditions

Emergency contact (Client): NLT Security Team security@nodelinotech.com

Backup contact (Client): NLT Operations ops@nodelinotech.com

System monitoring: Local Flask application logs and health endpoints (`/api/health`)

The Client may immediately **pause or stop** testing if:

- Service stability or availability is materially affected.
- Production data is at risk and immediate remediation is needed.
- An accidental exposure of sensitive production secrets occurs that requires immediate containment.

Tester will immediately pause testing if:

- A test produces unexpected broad impact beyond agreed scope.
- Ops requests an immediate stop (Tester must comply and document the reason).

Pause/stop command can be issued via email to security contact; the Tester must acknowledge within 2 hours during business hours, 4 hours outside business hours.

12. Evidence & Reporting Deliverables

Deliverables & timeline:

- **Daily status updates:** Brief summary of progress and significant findings via email during active testing
- **Draft technical report:** 5 business days after test completion — includes executive summary, findings with PoCs, AI-specific attack vectors, and remediation guidance
- **Final report:** Updated version after NLT feedback (1 revision cycle included)
- **Retest verification:** Follow-up testing to validate remediation of critical and high-risk findings (timeline to be agreed based on remediation schedule)

Contents of technical report:

- Executive summary with risk ratings.
 - Reproducible PoCs (sanitized).
 - Full request/response logs (securely stored).
 - Remediation recommendations and suggested acceptance criteria.
 - Estimated cost/effort to exploit findings (where applicable).
-

13. Legal & Liability

- Tester performs tests as an independent contractor. Client affirms authorization for the Tester as described and agrees to indemnify Tester for acts performed within this RoE.
 - Tester will not intentionally destroy data or otherwise perform illegal acts beyond the signed RoE.
 - Both parties agree to keep findings confidential. Public disclosure requires mutual written consent.
-

14. Acceptance Criteria & Success Metrics (NLT-specific)

Define acceptance as part of remediation verification. Example criteria the Client may require to close a finding:

- **AI Security Level Bypass:** Security levels 2-5 successfully block jailbreak attempts; verified by 50 prompt injection attempts across all security levels
 - **Harmful/Off-topic Output:** Security levels 2-5 successfully block attempts at generating harmful or off-topic content; verified by 50 attempts across all security levels
 - **RAG Information Disclosure:** Sensitive documents in `/knowledge_base/development/` are not retrievable by standard users; verified by 20 targeted queries for API keys and credentials
 - **Admin Interface Authorization:** Customer accounts cannot access admin endpoints; verified by authentication bypass attempts on all `/api/admin/*` routes
 - **AI Agent Manipulation:** Autonomous ticket decisions cannot be manipulated maliciously; verified by conversation injection attempts targeting ticket closure/escalation
 - **Session Security:** Session cookies use secure flags and expire properly; CSRF protection active on state-changing operations
 - **Vector Database Access:** ChromaDB accessible only via application API, not direct database access; verified by connection attempts
-

15. NLT System Architecture & Deployment Context

System Overview

- **Application:** Flask web application with AI chatbot functionality
- **AI Engine:** Ollama local LLM inference (various Mistral models)
- **Vector Database:** ChromaDB for RAG (Retrieval-Augmented Generation)
- **Session Store:** Server-side sessions with SQLite backend
- **Knowledge Base:** Markdown documents in `/knowledge_base/` (73+ documents)
- **Deployment:** Docker containerized with multiple deployment modes

Network Architecture

- **Web Interface:** Port 5000 (Flask application)
- **Ollama API:** Port 11434 (model inference, configurable)
- **Docker Network:** Internal container networking for Ollama ↔ Flask communication
- **External Access:** Web interface exposed, Ollama typically internal-only

Security Context

- **AI Security Levels:** 1-5 (configurable via `AI_SECURITY_LEVEL` environment variable)
 - Level 1: No filtering (educational/demo mode)
 - Level 2: Pattern-based jailbreak detection
 - Level 3: AI-powered input analysis with threat scoring
 - Level 4: Output content moderation only
 - Level 5: Full input + output filtering
- **Session Management:** 24-hour session lifetime, secure cookie flags
- **CSRF Protection:** Enabled for web forms, exempt for API endpoints
- **Rate Limiting:** Per-endpoint limits (login, chat, admin operations)
- **Role-Based Access:** Customer vs Admin roles with different privileges

Data Sensitivity

- **High Sensitivity:** Development credentials in `/knowledge_base/development/keys.md`
 - **Medium Sensitivity:** Customer conversations, support tickets, user accounts
 - **Low Sensitivity:** Product manuals, FAQs, general knowledge base content
 - **Synthetic Test Data:** Pre-loaded customer and admin accounts for testing
-

17. Contact & Sign-off

By signing below, NLT authorizes the Tester to perform security assessment activities against the NLT AI Chatbot system, within the time window and constraints listed above.

Tester (name, signature): _____ **Date:** _____

NLT Client (name, signature): _____ **Date:** _____ **NLT Technical**

Lead (name, signature): _____ **Date:** _____
